

Contadores, acumuladores, máximos y flags

Esteban Álvarez

Contadores, acumuladores, máximos y flags

- Una vez vistas las estructuras de control es necesario profundizar en una serie de **patrones que se repiten** en los programas
- Se trata del uso de
 - **Contadores**
 - **Acumuladores**
 - **Máximos**
 - **Flags** (banderas o centinelas)

Contadores

- Un contador es una **variable que cuenta** ocurrencias
- Su variante más simple es un contador que **cuenta cuántas veces se ha ejecutado un bucle**

```
int contador=0;
for(int i=0;i<3;i++) {
    contador++;
}
System.out.println("For con "+contador+" repeticiones");
```

Contadores

- Características **importantes**

```
int contador=0; //En este momento hay 0 repeticiones
for(int i=0;i<3;i++) {
    contador++;
}
System.out.println("For con "+contador+" repeticiones");
```

1. El contador se **inicializa** antes de entrar en el bucle
 - De no hacerlo así, el resultado será incorrecto
 - i. Prueba a inicializar `contador` dentro del `for` y analiza el resultado
 - El valor de la inicialización (0, 1, N) dependerá del problema

Contadores

- Características **importantes**

```
int contador=0; //En este momento hay 0 repeticiones
for(int i=0;i<3;i++) {
    contador++;
}
System.out.println("For con "+contador+" repeticiones");
```

2. El contador **actualiza su valor** dentro del bucle

- Cada vez que hay una iteración, sumamos 1 al contador
- La actualización de un contador **siempre suele ser ++** (o --)

Contadores

- Características **importantes**

```
int contador=0; //En este momento hay 0 repeticiones
for(int i=0;i<3;i++) {
    contador++;
}
```

```
System.out.println("For con "+contador+" repeticiones");
```

3. **Utilizamos** el valor almacenado en el contador después del bucle

- El caso más simple es mostrar el valor por pantalla
- Pero es muy habitual usar la **sentencia if-else**

```
if (contador!=0){...
```

Contadores

- En el caso de un for **no necesitamos un contador** de repeticiones, recuerda que utilizamos un for porque sabemos el número de repeticiones
- Pero en un **while** la cosa cambia

```
System.out.println("Introduce un número");  
int num=teclado.nextInt();  
while(num!=0) {  
    System.out.println("Introduce otro número");  
    num=teclado.nextInt();  
}
```

- ¿Cómo mostramos un mensaje al usuario indicando cuántos números ha introducido?
- **SOLUCIÓN:** contador

Contadores

```
System.out.println("Introduce un número");
int num=teclado.nextInt();
int contador=1; //CUIDADO!! El usuario ya ha introducido un número
while(num!=0) {
    System.out.println("Introduce otro número");
    num=teclado.nextInt();
    contador++; //Después de leer el número, incrementamos contador
}
System.out.println(contador+" números introducidos");
```


Contadores

- Otra utilidad un poco más avanzada de los contadores es **contar ocurrencias** de una determinada condición
- Se necesita usar la **sentencia if**
- Ejemplo: Pedir al usuario tres números

```
for (int i = 0; i < 3; i++) {  
    System.out.println("Introduce un número");  
    double num=teclado.nextDouble();  
}
```

- Mostrar **cuántos de esos números son positivos**
- **SOLUCIÓN:** contador de positivos

Contadores

- Ahora no incrementamos `contador` en cada iteración. Lo haremos **solo si el número es positivo**

```
int positivos=0; //RECUERDA: identificadores descriptivos
for (int i = 0; i < 3; i++) {
    System.out.println("Introduce un número");
    double num=teclado.nextDouble();
    if (num>0) {
        positivos++;
    }
}
System.out.println("Hay "+positivos+" números positivos");
```

Contadores

- En el ejemplo anterior **no hay else** porque si el número es negativo no quiero hacer nada especial
- Programa que cuente números positivos y negativos:

```
int positivos=0;
int negativos=0; //Necesito un contador nuevo
for (int i = 0; i < 3; i++) {
    System.out.println("Introduce un número");
    double num=teclado.nextDouble();
    if (num>0) {
        positivos++;
    }else {
        negativos++;
    }
}
System.out.println("Hay "+positivos+" positivos y "+negativos+" negativos");
```

Acumuladores

- Se comportan igual que los contadores pero en lugar de sumar 1 suelen **sumar o restar otras cantidades**
- Se utilizan para la **suma acumulativa** de cantidades
- Ejemplo de total acumulado de litros de lluvia de la semana:

Día	Litros	Acumulado	Explicación
Lunes	2	$2 + 0 = 2$	2 litros del lunes + 0 litros acumulados=2
Martes	1	$1 + 2 = 3$	Sumamos 1 litro del martes a los anteriores (Lunes)
Miércoles	0	$0 + 3 = 3$	0 litros del Miércoles + los anteriores (Lunes y Martes)
Jueves	3	$3 + 3 = 6$	3 litros del Jueves + los anteriores (L,M,X)
Viernes	2	$2 + 6 = 8$	2 litros del Viernes+ los anteriores (L,M,X,J)
Sábado	1	$1 + 8 = 9$	1 litro del sábado + los anteriores (L,M,X,V)
Domingo	0	$0 + 9 = 9$	0 litros del domingo + los anteriores (L,M,X,V,S)

Acumuladores

- Cada día se **repite la operación**:

`litrosTotales=litrosTotales+litrosDelDía;`

- Los litros acumulados hasta **en ese día** **Jueves**
- Son los **litros que había** acumulados en los días anteriores **L,M,X**
- **+ los litros del día actual** **Jueves**

Acumuladores

- Recuerda que esto:

```
litrosTotales=litrosTotales+litrosDelDía;
```

- Es equivalente a

```
litrosTotales+=litrosDelDía;
```

El **operador** típico de un acumulador es += (-= *= /= ...)

Acumuladores

EJERCICIO: Programa que lea 5 notas por teclado y obtenga la media

Algoritmo

Pedir 5 notas

Ir acumulando la suma de notas

$\text{Media} = \text{Suma de notas} / \text{n}^\circ \text{ de notas}$

Acumuladores

EJERCICIO: Programa que lea 5 notas por teclado y obtenga la media

```
double sumaNotas=0; //Al inicio la suma de notas es 0
for (int i = 0; i < 5; i++) {
    System.out.println("Introduce una nota");
    double nota=teclado.nextDouble();
    sumaNotas+=nota; //Acumulamos la suma de notas
}
System.out.println("La media es: "+ sumaNotas/5);
```

Inicializar antes del bucle, **actualizar** dentro del bucle y **utilizar** el resultado después

Máximos (y mínimos)

- Otro problema recurrente es el de **localizar el máximo** (o mínimo) de entre un conjunto de valores, habitualmente números enteros o reales
- El algoritmo que usamos los humanos consiste en ver el total de los valores y localizar visualmente el mayor de ellos

5 1 8 3 9 0 7

Máximos (y mínimos)

- Pero ese algoritmo no se puede aplicar en Java porque **no vemos a la vez todos los valores**
- La solución consiste en ir viendo cuál es el valor máximo **hasta el momento**



Máximos (y mínimos)

- Por tanto necesitamos una variable `maximo` que vaya guardando el valor mayor hasta ese momento
- Al finalizar la ejecución, `máximo` almacena el mayor de todos los valores

Máximos (y mínimos)

- Problema: Programa que pida 5 números al usuario y diga cuál es el mayor.

Algoritmo

Para cada número introducido

 Mirar a ver si es el mayor hasta el momento

Mostrar máximo

Máximos (y mínimos)

- Ejemplo: Programa que pida 5 números al usuario y diga cuál es el mayor.

```
//IMPORTANTE: Hay que inicializar la variable con un valor
//que sepamos que se va a superar
int maximo=-10000;
for(int i=0;i<5;i++) {
    System.out.println("Introduce un número");
    int num=sc.nextInt();
    //Si el número actual es mayor que el que había
    if(num>maximo) {
        //El número actual es el mayor, por el momento
        maximo=num;
    }
}
System.out.println("El mayor número introducido es "+maximo);
```

Flags

- Los flags son **variables booleanas** que cambian su valor cuando se produce un determinado evento o situación
- EJERCICIO: Programa que pida 5 notas al usuario y muestre si hay (o no) algún suspenso. Para ello debe mostrar el mensaje “Hay algún suspenso” o “No hay ningún suspenso”
 - Implementa una solución utilizando **contadores** (con un contador de suspensos)

Flags

- Implementa una solución utilizando **contadores** (con un contador de suspensos)

```
int suspensos=0; //Inicialización
for (int i = 0; i < 5; i++) {
    System.out.println("Introduce una nota");
    double nota=teclado.nextDouble();
    if (nota<5) { //Actualización
        suspensos++;
    }
}
if(suspensos>0){ //Resultados. suspensos puede almacenar 0,1,2,3,4,5
    System.out.println("Hay suspensos");
}else {
    System.out.println("No hay suspensos");
}
```

Flags

- Pero la **mejor solución** es utilizar una variable booleana que cambie de `false`→`true` cuando se detecte un suspenso

Algoritmo

```
haySuspendos=false
```

```
Pedir 5 notas
```

```
    cuando se detecte un suspenso, haySuspendos=true
```

```
Mostrar si hay o no suspendos
```


Flags

```
boolean haySuspensos=false;
for (int i = 0; i < 5; i++) {
    System.out.println("Introduce una nota");
    double nota=teclado.nextDouble();
    if (nota<5) { //Cuando se produce un evento
        haySuspensos=true; //Activo la bandera
    }
}
//haySuspensos funciona como un "chivato" que
if(haySuspensos){ //me indica si ha ocurrido o no un evento
    System.out.println("Hay suspensos");
}else {
    System.out.println("No hay suspensos");
}
```

Poner aquí un **else** es un ERROR GRAVE: si hay un suspenso y la variable se activa (`true`) no quiero que cuando detecte un aprobado se desactive (`false`)

```
boolean haySuspensos=false;
for (int i = 0; i < 5; i++) {
    System.out.println("Introduce una nota");
    double nota=teclado.nextDouble();
    if (nota<5) { //Cuando se produce un evento
        haySuspensos=true; //Activo la bandera
    }else {
        haySuspensos=false;
    }
}
if(haySuspensos){
    System.out.println("Hay suspensos");
}else {
    System.out.println("No hay suspensos");
}
```

FIN!

(Pasar a la presentación sobre el Ámbito de las variables)